SEMT30008 Week 21 Lab - LLM Post-Training: From Alignment to Reasoning

Introduction

Welcome to the LLM Post-Training Lab!

In this 2-hour session, we will demystify the post-training pipeline that turns a raw "document continuator" into an aligned, autonomous reasoning agent like ChatGPT, o1, or DeepSeek-R1. Based on our lecture, we will implement the core mathematical and algorithmic components of the post-training pillars:

- Fine-Tuning Foundations (SFT, PEFT)
- Alignment & Human Preferences (RLHF, Reward Modeling, PPO, DPO)
- Reasoning-Centric RL (Latent Reasoning, MDP, GRPO)
- Test-Time Scaling (Self-Consistency, Best-of-N, Adaptive Compute)
- Efficiency & Open Problems (Merging, Reward Hacking)

Let's dive in! Run the setup cell below to import necessary libraries.

In [ ]:
```
# Run this first!
import re
import math
import torch
import torch.nn as nn
import torch.nn.functional as F
import numpy as np
from collections import Counter
import matplotlib.pyplot as plt
```

Module 1: Fine-Tuning Foundations

Reference: Slides 11–16

Supervised Fine-Tuning (SFT) transitions a model into an "instruction follower". Because full-parameter fine-tuning is extremely expensive, the industry relies on Parameter-Efficient Fine-Tuning (PEFT) like LoRA.

Exercise 1: SFT Data Formatting

Data quality is king in SFT. Before feeding data to the model, we must wrap it in a strict prompt template so the model learns the "format and style" of an assistant.

Task: Format a raw dataset row into a standard SFT conversational string.

In [ ]:
```
def format_sft_example(instruction, response):
    # TODO 1: Use an f-string to format the input as:
    # "User: [instruction]\nAssistant: [response]<|endoftext|>"
    formatted_text = f"User: {___}\nAssistant: {___}<|endoftext|>"
    return formatted_text

inst = "What is the capital of France?"
resp = "The capital of France is Paris."
print(format_sft_example(inst, resp))
```

Module 2: Alignment & Human Preferences

Reference: Slides 17–32

Alignment ensures models are Helpful, Honest, and Harmless. We use Reinforcement Learning from Human Feedback (RLHF) and Direct Preference Optimization (DPO).

Exercise 2: Reward Modeling

A Reward Model $R_\theta(x, y)$ learns human preferences by assigning a higher scalar score to a preferred response (y_1) over a rejected one (y_2).

Task: Implement the Bradley-Terry probability of choosing y_1 over y_2 using the sigmoid of their difference.

In [ ]:
```
def bradley_terry_prob(reward_chosen, reward_rejected):
    # TODO 4: Calculate the probability using torch.sigmoid on the difference
    diff = ___ - ___
    prob = torch.sigmoid(diff)
    return prob.item()

r_y1 = torch.tensor([2.5]) # High reward
r_y2 = torch.tensor([-1.2]) # Low reward
print(f"Probability of picking y1 over y2: {bradley_terry_prob(r_y1, r_y2):.4f
```

Exercise 3: PPO Trust Region (Clipping)

Standard RL is highly unstable for language (Slide 22). PPO ensures the new policy doesn't move too far from the old policy using a clipping mechanism.

Task: Clip the probability ratio between the bounds $[1 - \epsilon, 1 + \epsilon]$.

In [ ]:
```
def ppo_clip(ratio, epsilon=0.2):
    # TODO 5: Use torch.clamp to restrict the ratio between 1-epsilon and 1+ep
    clipped_ratio = torch.clamp(ratio, ___, ___)
    return clipped_ratio

prob_ratio = torch.tensor([1.5]) # Policy shifted massively
print(f"Original Ratio: {prob_ratio.item()} -> Clipped: {ppo_clip(prob_ratio).
```

Exercise 4: KL-Divergence Penalty

To prevent "Reward Hacking" (outputting gibberish that scores high), RLHF anchors the RL model to the original SFT model using a KL penalty (Slide 24).

Task: Compute the KL penalty given the log probabilities of the RL policy and the Reference (SFT) policy.

In [ ]:
```
def kl_penalty(log_pi_rl, log_pi_ref, beta=0.1):
    # TODO 6: Calculate the KL divergence penalty: beta * (log_pi_rl - log_pi_
    penalty = ___
    return penalty

log_pi_rl = torch.tensor([-0.5])
log_pi_sft = torch.tensor([-2.0]) # RL model diverges from SFT
print(f"KL Penalty subtracted from reward: {kl_penalty(log_pi_rl, log_pi_sft).
```

Exercise 5: DPO (Direct Preference Optimization) Loss

DPO bypassed the complex Reward Model entirely. It increases the likelihood of the chosen response and decreases the likelihood of the rejected response.

Task: Implement the DPO loss function core logic.

In [ ]:
```
def dpo_loss(pi_logps_chosen, ref_logps_chosen, pi_logps_rej, ref_logps_rej, b
    # TODO 7: Calculate the implicit reward for chosen and rejected
    pi_ratio_chosen = pi_logps_chosen - ref_logps_chosen
    pi_ratio_rej = pi_logps_rej - ref_logps_rej

    # TODO 8: Calculate the loss using negative log sigmoid
    loss = -F.logsigmoid(beta * (___ - ___))
    return loss.item()
```

```
# Dummy logprobs
print(f"DPO Loss: {dpo_loss(0.9, 0.8, -1.2, -0.5):.4f}")
```

Module 3: Reasoning-Centric Post-Training

Reference: Slides 33–41

Models like DeepSeek-R1 and o1 use RL to discover reasoning. We treat reasoning as a Markov Decision Process (MDP).

Exercise 6: Parsing Latent Reasoning (The `<think>` tag)

Reasoning agents plan and self-correct in a hidden "scratchpad" before answering.

Task: Extract the latent thinking process and the final answer using Regex.

In [ ]:
```
def parse_reasoning_output(text):
    # TODO 9: Write a regex to capture text inside <think>...</think>
    think_match = re.search(r'<think>(.*?)</think>', text, ___)
    think_process = think_match.group(1).strip() if think_match else ""

    # TODO 10: Extract the final answer (everything after </think>)
    final_answer = text.split("</think>")[-1].strip()
    return think_process, final_answer

output = "<think>\nWait, 2+2 is 4.\n</think>\nThe answer is 4."
think, ans = parse_reasoning_output(output)
print(f"Thought: {think}\nAnswer: {ans}")
```

Exercise 7: Dense (Process) Reward in MDP

Sparse rewards (1 at the end if correct) cause the "Credit Assignment Problem". Process Reward Models (PRM) give dense points for each valid intermediate step.

Task: Calculate the total reward for a reasoning trajectory.

In [ ]:
```
def calculate_process_reward(steps, is_final_correct):
    total_reward = 0.0
    # TODO 11: Add 0.1 for every step in the 'steps' list
    for step in steps:
        total_reward += ___

    # TODO 12: Add a sparse reward of 1.0 if the final answer is correct
    if is_final_correct:
        total_reward += ___

    return total_reward
```

```
reasoning_chain = ["x = 5", "y = 10", "x + y = 15"]
print(f"Total MDP Reward: {calculate_process_reward(reasoning_chain, True)}")
```

Exercise 8: GRPO (Group Relative Policy Optimization)

DeepSeek-R1 skips the massive "Critic" model in PPO. Instead, it generates a group of outputs, scores them, and normalizes the rewards so the model learns by comparing its own variations.

Task: Normalize a group of rewards to have a mean of 0 and std of 1.

In [ ]:
```
def grpo_normalize(rewards):
    rewards_tensor = torch.tensor(rewards, dtype=torch.float32)
    # TODO 13: Calculate mean and standard deviation
    mean_r = rewards_tensor.___()
    std_r = rewards_tensor.___()

    # TODO 14: Normalize the rewards (add 1e-8 to std to prevent div by zero)
    norm_rewards = (rewards_tensor - mean_r) / (std_r + 1e-8)
    return norm_rewards.tolist()

group_rewards =[0.2, 0.5, 0.1, 0.9, 0.0] # 5 sampled paths
print([round(r, 2) for r in grpo_normalize(group_rewards)])
```

Module 4: Test-Time Scaling & Inference Compute

Reference: Slides 42–51

Instead of just scaling training, we scale inference compute (Test-Time Scaling).

Exercise 9: Self-Consistency (Majority Voting)

Generate N different reasoning paths. Pick the answer that appears most frequently.

Task: Implement Majority Voting to find the most robust answer.

In [ ]:
```
def majority_vote(answers):
    # TODO 15: Use collections.Counter to find the most common answer
    counts = Counter(___)
    best_answer = counts.most_common(1)[0][0]
    return best_answer

rollouts = ["4", "5", "4", "Error", "4", "5"]
print(f"Majority Voted Answer: {majority_vote(rollouts)}")
```

Exercise 10: Best-of-N (Rejection Sampling)

Generate N responses and pass them through a Reward Model. Select the one with the highest score.

Task: Write a function to extract the string with the maximum reward score.

In [ ]:
```python
def best_of_n(responses_with_scores):
    # TODO 16: Return the response text that has the maximum 'score'
    # Hint: Use the max() function with a lambda key
    best = max(responses_with_scores, key=lambda x: ___)
    return best["text"]

samples =[
    {"text": "The answer is basically 5.", "score": 0.4},
    {"text": "Let's think step-by-step. 2+3=5.", "score": 0.9},
    {"text": "5.", "score": 0.2}
]
print(f"Best-of-N Selection: {best_of_n(samples)}")
```

Exercise 11: Adaptive Test-Time Compute (Routing)

Running MCTS or Best-of-100 on every prompt wastes energy. We act as a "smart thermostat" based on task difficulty.

Task: Route the prompt to the correct TTS strategy based on its difficulty.

In [ ]:
```python
def adaptive_tts_router(difficulty_score):
    # TODO 17: Route logic based on Slide 49
    # score < 0.3 -> "Direct Generation"
    # score < 0.7 -> "Self-Refine"
    # else -> "Parallel Beam Search / MCTS"
    if difficulty_score < 0.3:
        return ___
    elif difficulty_score < 0.7:
        return ___
    else:
        return ___

print(f"Hard Math Question (0.9): {adaptive_tts_router(0.9)}")
```

Exercise 12: Model Merging (Task Arithmetic)

We can combine the skills of an aligned model and a math expert model without gradient updates using weight interpolation.

In [ ]:
```python
def merge_models(W_base, W_task, alpha=0.5):
    # TODO 18: Implement Task Arithmetic formula: W_merged = W_base + alpha *
```

```
    # Alternatively: (1 - alpha) * W_base + alpha * W_task
    W_merged = ___
    return W_merged

W_b = torch.tensor([1.0, 1.0])
W_t = torch.tensor([3.0, 3.0])
print(f"Merged Weights (alpha=0.5): {merge_models(W_b, W_t, 0.5).tolist()}")
```

Exercise 13: RAG Prompt formatting (Domain Adaptation)

Retrieval-Augmented Generation (RAG) injects specialized knowledge at test-time.

Task: Construct a RAG prompt injecting retrieved clinical chunks.

In [ ]:
```
def construct_rag_prompt(user_query, retrieved_chunks):
    context = "\n".join(retrieved_chunks)
    # TODO 19: Build a prompt asking the model to answer the query ONLY using
    prompt = f"Context:\n{___}\n\nQuery: {___}\nAnswer strictly based on the c
    return prompt

docs =["Symptom X is treated with Drug Y.", "Drug Y causes drowsiness."]
print(construct_rag_prompt("How do I treat Symptom X?", docs))
```

Exercise 14: Open Problems - Mitigating Reward Hacking

Reward Models often suffer from "verbosity bias" (Slide 64), secretly learning that "longer is better".

Task: Implement a dynamic penalty that reduces the reward if the model overthinks or generates too many tokens unnecessarily.

In [ ]:
```
def penalize_verbosity(base_reward, token_length, max_allowed_tokens=500, pena
    # TODO 20: If token_length > max_allowed_tokens, subtract (excess_tokens *
    if token_length > max_allowed_tokens:
        excess = ___ - ___
        base_reward -= (excess * ___)
    return base_reward

score = 0.95
print(f"Score for 100 tokens: {penalize_verbosity(score, 100)}")
print(f"Score for 1000 tokens (Reward Hacked): {penalize_verbosity(score, 1000
```